

1) Pesquisar um código de outro autor que aplica A* para resolver algum problema (jogos, rotas, etc). Citar a fonte e link do código original!

Fonte do código utilizado: <https://github.com/malufreitas/a-estrela/>

a) Executar o código e anexar um print da saída.

```
C:\Users\beatr\OneDrive\Área de Trabalho\UNIFESP - 2026.1\IA\A-estrela> python main.py teste.txt
Exemplo de entrada (x,y): = '0 0'

Entre com o ponto inicial do trajeto: 0 0
Entre com o ponto final do trajeto: 4 4
### PERCURSO de (0, 0) ate (4, 4) ###
[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4), (4, 4)]

[['A', '>', '>', '>', '>'], [0, 1, 0, 1, 'v'], [0, 0, 0, 1, 'v'], [0, 1, 0, 0, 'v'], [0, 0, 0, 0, 'B']]

[['A', '>', '>', '>', '>'], ['-', '■', '-', '■', 'v'], ['-', '-', '-', '■', 'v'], ['-', '■', '-', '-', 'v'], ['-', '-', '-', '-', 'B']]
A > > >
- ■ - ■ v
- - - ■ v
- ■ - - v
- - - - B
```

b) Explicar brevemente como o código resolve o problema.

O código tem como objetivo encontrar o menor caminho entre dois pontos, a partir da entrada recebida pelo usuário, através do algoritmo de A*.

Seu funcionamento inicia criando um mapa, em forma de matriz, a partir de um arquivo .txt definido. Onde a matriz é composta por: 0 - posição livre e 1 - obstáculo.

O algoritmo organiza três estruturas sendo:

- listaAberta: armazenando os nós a serem explorados
- listaFechada: nós que já foram visitados
- dcPosicoesCalculadas: guarda informações de cada nó visitado.

Assim, a cada posição é armazenado: (coordenada, custo, heurística, pai). Sendo o custo a distância do início até o nó atual, a heurística uma estimativa até o objetivo e o pai, o nó anterior, usado para reconstruir o caminho

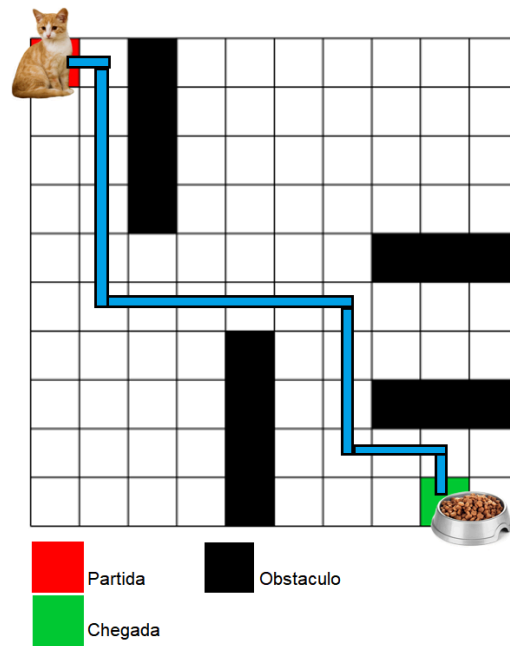
A principal função, que controla o algoritmo de A* é a buscar(), que a cada iteração pega o primeiro elemento de listaAberta e verifica seus vizinhos válidos, aqueles que não são obstáculos, não foram visitados e estão dentro do mapa.

Para cada vizinho, é calculado o custo, através do custo acumulado ($g(n)$), a heurística de Manhattan ($h(n)$) e por fim, para ordenação utiliza a soma de ambos ($f(n)$).

Após o cálculo do custo, os vizinhos são adicionados na listaAberta, e o nó atual na listaFechada, atualizando as posições visitadas. A listaAberta é ordenada pelo menor valor de $f(n)$, garantindo que o algoritmo explore sempre o caminho de menor custo.

O algoritmo então é encerrado quando o nó final (informado pelo usuário) é encontrado, e assim o caminho percorrido é reconstruído, percorrendo os pais de cada nó até chegar à posição inicial.

A exibição final mostra o caminho encontrado, sendo o início A e final B, representando o percurso com setas e símbolos para posição livre/obstáculo.



c) Analisar o código e descrever qual heurística foi aplicada.

A heurística utilizada no código é a **distância de Manhattan**, definida por:

$$h(n) = |x_{atual} - x_{objetivo}| + |y_{atual} - y_{objetivo}|$$

Essa heurística considera apenas movimentos horizontais e verticais, não superestima o custo real e garante que o A* encontre o caminho ótimo.

d) Se os autores comparam a busca A* com outra busca clássica, etc. Mostrar o resultado comparativo. O autor não apresentou comparação nos métodos de busca.